部署管理画面UI設計書

メタデータ

title: 部署管理画面UI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-01-23
author: Development Team
tags: [system-admin, department, list, main]

関連ドキュメント

- 部署追加ダイアログ
- 部署編集ダイアログ
- マスタ管理API仕様
- 部署テーブルDDL
- バックエンド実装計画
- UIポリシー

---

2. 画面レイアウト

2.1. 画面イメージ



---

3. 概要

3.1. 目的

システム管理者が部署マスタを管理するためのメイン画面を定義します。部署の一覧表示、検索、フィルタリング、追加、編集、ステータス切替、CSV入出力機能を提供します。

3.2. 利用権限

- **System Admin（システム管理者）**: 使用可能

3.3. URL

`/system-admin/department`

---

2. 画面構成

2.1. レイアウト

```
┌──────────────────────────────────────────────────────────┐
│ システム管理者 > 部署管理                                │
├──────────────────────────────────────────────────────────┤
```



---

3. ツールバー

3.1. 配置

左側: 検索・フィルタUI **右側**: CSV操作、追加ボタン（右から主要な順）

3.2. コンポーネント

3.2.1. 検索入力

コンポーネント: `<Input>`

仕様:

項目	値
プレースホルダー	「部署CD、部署名で検索」
前置アイコン	`<SearchOutlined />`
幅	250px
クリアボタン	有効（allowClear）

検索対象:

- 部署CD（部分一致）
- 部署名（部分一致）

3.2.2. ステータスフィルター

コンポーネント: `<Select>`

仕様:

項目	値
プレースホルダー	「ステータス」
幅	150px
クリアボタン	有効（allowClear）

選択肢:

- すべて（フィルタなし）
- アクティブ
- 非アクティブ

3.2.3. CSVダウンロードボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「CSV」
タイプ	`default`
アイコン	`<DownloadOutlined />`
サイズ	省略（デフォルト）

動作:

1. 現在のフィルタ・検索条件を適用
2. API呼び出し: `GET /api/v1/master/department/download-csv`
3. CSVファイルをダウンロード

4. ファイル名: `department_YYYYMMDD_HHmmss.csv`

3.2.4. CSVアップロードボタン

コンポーネント: `<Upload>` + `<Button>`

仕様:

項目	値
ラベル	「CSV一括登録」
タイプ	`default`
アイコン	`<UploadOutlined />`
サイズ	省略（デフォルト）
許可形式	.csv

動作:

1. CSVファイル選択
2. アップロード前にプレビュー表示（オプション）
3. API呼び出し: `POST /api/v1/master/department/upload-csv`
4. 結果表示（成功件数、失敗件数、エラー詳細）

3.2.5. 追加ボタン

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「追加」
タイプ	`primary`
アイコン	`<PlusOutlined />`
サイズ	省略（デフォルト）

動作:

1. 部署追加ダイアログを開く

2. ダイアログで部署情報を入力
3. 保存後、一覧を再取得

---

4. データテーブル

4.1. テーブル仕様

コンポーネント: `<Table>`

主要設定:

項目	値
rowKey	`departmentCd`
pagination	有効
pageSize	20（デフォルト）
pageSizeOptions	[10, 20, 50, 100]
scroll	`{ x: 1000 }`

4.2. カラム定義

4.2.1. 部署CD

プロパティ:

項目	値
title	「部署CD」
dataIndex	`departmentCd`
key	`departmentCd`
width	150px
align	`left`
sorter	有効

4.2.2. 部署名

プロパティ:

項目	値
title	「部署名」
dataIndex	`departmentName`
key	`departmentName`
width	250px
align	`left`
sorter	有効

4.2.3. 状態

プロパティ:

項目	値
title	「状態」
dataIndex	`isActive`
key	`isActive`
width	120px
align	`center`
sorter	有効

レンダリング:

```
render: (isActive: boolean) => (
  <Badge
    status={isActive ? 'success' : 'default'}
    text={isActive ? 'アクティブ' : '非アクティブ'}
  />
)
```

4.2.4. 更新日時

プロパティ:

項目	値
title	「更新日時」
dataIndex	`updateAt`
key	`updateAt`
width	180px
align	`right`
sorter	有効
defaultSortOrder	`descend`

レンダリング:

```
render: (date: string) => {
  return date ? new Date(date).toLocaleString('ja-JP') : '-';
}
```

4.2.5. 操作

プロパティ:

項目	値
title	「操作」
key	`action`
width	150px
align	`center`
fixed	`right`

レンダリング:

```
render: (_, record) => (
  <Space size="small">
    <Button
      type="default"
      size="small"
```

```
      icon={<EditOutlined />}
      onClick={() => handleEdit(record.departmentCd)}
    >
      編集
    </Button>
    <Button
      type="default"
      size="small"
      style={record.isActive ? {
        color: token.colorError,
        borderColor: token.colorError
      } : {
        color: token.colorSuccess,
        borderColor: token.colorSuccess
      }}
      onClick={() => handleStatusToggle(record.departmentCd, !record.isActive)}
    >
      {record.isActive ? '非アクティブ化' : 'アクティブ化'}
    </Button>
  </Space>
)
```

---

5. ページネーション

5.1. 仕様

コンポーネント: `<Table>` の pagination プロパティ

設定:

```
pagination={{
  pageSize: 20,
  showSizeChanger: true,
  showTotal: (total) => `全 ${total} 件`,
  pageSizeOptions: ['10', '20', '50', '100']
}}
```

---

6. APIリクエスト仕様

6.1. 一覧取得

エンドポイント: `GET /api/v1/master/department`

クエリパラメータ:

パラメータ	型	説明
search	string	検索テキスト（部署CD、部署名）
is_active	boolean	アクティブフィルタ
sort_by	string	ソートフィールド（department_cd, department_name, update_at）
sort_order	string	ソート順（asc, desc）
page	number	ページ番号（デフォルト: 1）
limit	number	1ページあたりの件数（デフォルト: 20）

リクエスト例:

```
GET /api/v1/master/department?search=研究
&is_active=true&sort_by=update_at&sort_order=desc&page=1&limit=20
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

レスポンス:

```
{
  "success": true,
  "data": [
    {
      "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
      "departmentCd": "DEPT001",
      "departmentName": "研究開発部",
      "isActive": true,
      "createAt": "2026-01-20T10:30:00.000Z",
      "updateAt": "2026-01-23T10:30:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "totalItems": 25,
    "totalPages": 2,
```

```
    "hasNext": true,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-01-23T10:30:00.000Z",
    "requestId": "req_abc123"
  }
}
```


6.2. ステータス更新

エンドポイント: `PATCH /api/v1/master/department/:departmentCd/status`

リクエストボディ:


```
{
  "isActive": false
}
```


レスポンス:


```
{
  "success": true,
  "data": {
    "organizationId": "9eea5300-db8b-4b2d-8e9e-ec306c476b12",
    "departmentCd": "DEPT001",
    "departmentName": "研究開発部",
    "isActive": false,
    "createAt": "2026-01-20T10:30:00.000Z",
    "updateAt": "2026-01-23T11:00:00.000Z"
  },
  "meta": {
    "timestamp": "2026-01-23T11:00:00.000Z",
    "requestId": "req_def456"
  }
}
```


6.3. CSVダウンロード

エンドポイント: `GET /api/v1/master/department/download-csv`

クエリパラメータ: 一覧取得と同じ（search, is_active）

レスポンス: CSVファイル（UTF-8 BOM付き）

CSV形式:

```
部署CD,部署名,状態
DEPT001,研究開発部,アクティブ
DEPT002,品質管理部,アクティブ
DEPT003,総務部,非アクティブ
```

6.4. CSVアップロード

エンドポイント: `POST /api/v1/master/department/upload-csv`

リクエストヘッダー:

```
Content-Type: multipart/form-data
Authorization: Bearer {jwt_token}
x-organization-id: {organization_id}
```

レスポンス:

```
{
  "success": true,
  "data": {
    "successCount": 18,
    "failureCount": 2,
    "errors": [
      {
        "row": 5,
        "departmentCd": "DEPT999",
        "error": "部署CDが重複しています"
      },
      {
        "row": 12,
        "departmentCd": "",
        "error": "部署CDは必須です"
      }
    ]
  },
  "meta": {
```

```
    "timestamp": "2026-01-23T11:00:00.000Z",
    "requestId": "req_ghi789"
  }
}
```

---

7. イベントハンドリング

7.1. 検索

トリガー: 検索入力の変更（デバウンス500ms）

処理:

1. 検索テキストを状態に保存
2. 500msのデバウンス
3. API呼び出し（page=1にリセット）

7.2. ステータスフィルター

トリガー: ステータス選択の変更

処理:

1. 選択値を状態に保存
2. API呼び出し（page=1にリセット）

7.3. ソート

トリガー: カラムヘッダークリック

処理:

1. ソートフィールドとソート順を状態に保存
2. API呼び出し（page維持）

7.4. ページ変更

トリガー: ページネーションクリック

処理:

1. ページ番号を状態に保存
2. API呼び出し

7.5. 編集ボタンクリック

処理:

1. 部署CDを保存
2. 編集ダイアログを開く
3. 保存後、一覧を再取得

7.6. ステータス切替ボタンクリック

処理:

1. 確認ダイアログ表示（非アクティブ化の場合）
2. API呼び出し: `PATCH /api/v1/master/department/:departmentCd/status`
3. 成功時、一覧を再取得
4. 失敗時、エラーメッセージ表示

---

8. 実装例（TypeScript + Next.js + Ant Design）

```
// frontend/src/app/system-admin/department/page.tsx
'use client';

import React, { useState, useEffect } from 'react';
import {
  Table,
  Input,
  Select,
  Button,
  Space,
  Badge,
  message,
```

```
  Modal,
  Upload,
  theme
} from 'antd';
import {
  SearchOutlined,
  PlusOutlined,
  EditOutlined,
  DownloadOutlined,
  UploadOutlined
} from '@ant-design/icons';
import type { ColumnsType, TablePaginationConfig } from 'antd/es/table';
import apiClient from '@/utils/apiClient';
import DepartmentAddDialog from '@/components/SystemAdmin/DepartmentAddDialog';
import DepartmentEditDialog from '@/components/SystemAdmin/DepartmentEditDialog';

const { useToken } = theme;

interface Department {
  organizationId: string;
  departmentCd: string;
  departmentName: string;
  isActive: boolean;
  createAt: string;
  updateAt: string;
}

export default function DepartmentManagementPage() {
  const { token } = useToken();
  const [data, setData] = useState<Department[]>([]);
  const [loading, setLoading] = useState(false);
  const [searchText, setSearchText] = useState('');
  const [statusFilter, setStatusFilter] = useState<boolean | undefined>(undefined);
  const [sortField, setSortField] = useState('update_at');
  const [sortOrder, setSortOrder] = useState<'asc' | 'desc'>('desc');
  const [pagination, setPagination] = useState({
    current: 1,
    pageSize: 20,
    total: 0
  });

  const [isAddDialogOpen, setIsAddDialogOpen] = useState(false);
  const [isEditDialogOpen, setIsEditDialogOpen] = useState(false);
  const [editingDepartmentCd, setEditingDepartmentCd] = useState<string | null>(null);

  const organizationId = '9eea5300-db8b-4b2d-8e9e-ec306c476b12'; // 実際は認証情報から取得

  // データ取得
  const fetchData = async () => {
    setLoading(true);
```

```
    try {
      const response = await apiClient.get('/api/v1/master/department', {
        params: {
          search: searchText || undefined,
          is_active: statusFilter,
          sort_by: sortField,
          sort_order: sortOrder,
          page: pagination.current,
          limit: pagination.pageSize
        },
        headers: {
          'x-organization-id': organizationId
        }
      });

      if (response.data.success) {
        setData(response.data.data);
        setPagination({
          ...pagination,
          total: response.data.pagination.totalItems
        });
      }
    } catch (error: any) {
      message.error('データの取得に失敗しました');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchData();
  }, [searchText, statusFilter, sortField, sortOrder, pagination.current,
pagination.pageSize]);

  // 検索（デバウンス）
  useEffect(() => {
    const timer = setTimeout(() => {
      setPagination({ ...pagination, current: 1 });
    }, 500);
    return () => clearTimeout(timer);
  }, [searchText]);

  // ステータス切替
  const handleStatusToggle = (departmentCd: string, newStatus: boolean) => {
    const action = newStatus ? 'アクティブ化' : '非アクティブ化';
    Modal.confirm({
      title: `部署を${action}しますか？`,
      content: `${departmentCd} を${action}します。`,
      okText: 'はい',
      cancelText: 'いいえ',
```

```
      onOk: async () => {
        try {
          await apiClient.patch(
            `/api/v1/master/department/${departmentCd}/status`,
            { isActive: newStatus },
            {
              headers: {
                'x-organization-id': organizationId
              }
            }
          );
          message.success(`部署を${action}しました`);
          fetchData();
        } catch (error: any) {
          message.error(`${action}に失敗しました`);
        }
      }
    });
  };

  // CSVダウンロード
  const handleDownloadCsv = async () => {
    try {
      const response = await apiClient.get(
        '/api/v1/master/department/download-csv',
        {
          params: {
            search: searchText || undefined,
            is_active: statusFilter
          },
          headers: {
            'x-organization-id': organizationId
          },
          responseType: 'blob'
        }
      );

      const blob = new Blob([response.data], { type: 'text/csv' });
      const url = window.URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.href = url;
      link.download = `department_${new Date().toISOString().replace(/[:.]/g, '-')}.csv`;
      document.body.appendChild(link);
      link.click();
      document.body.removeChild(link);
      window.URL.revokeObjectURL(url);

      message.success('CSVをダウンロードしました');
    } catch (error) {
      message.error('CSVダウンロードに失敗しました');
```

```
  }
};

// CSVアップロード
const handleUploadCsv = async (file: File) => {
  const formData = new FormData();
  formData.append('file', file);

  try {
    const response = await apiClient.post(
      '/api/v1/master/department/upload-csv',
      formData,
      {
        headers: {
          'x-organization-id': organizationId,
          'Content-Type': 'multipart/form-data'
        }
      }
    );

    if (response.data.success) {
      const { successCount, failureCount, errors } = response.data.data;

      if (failureCount === 0) {
        message.success(`${successCount}件のデータを登録しました`);
      } else {
        Modal.warning({
          title: 'CSV登録結果',
          content: (
            <div>
              <p>成功: {successCount}件</p>
              <p>失敗: {failureCount}件</p>
              {errors.length > 0 && (
                <div>
                  <p>エラー詳細:</p>
                  <ul>
                    {errors.map((err: any, idx: number) => (
                      <li key={idx}>
                        行{err.row}: {err.error}
                      </li>
                    ))}
                  </ul>
                </div>
              )}
            </div>
          )
        });
      }
      fetchData();
    }
```

```
    } catch (error: any) {
      message.error('CSVアップロードに失敗しました');
    }

    return false; // prevent default upload behavior
  };

  // テーブルカラム定義
  const columns: ColumnsType<Department> = [
    {
      title: '部署CD',
      dataIndex: 'departmentCd',
      key: 'departmentCd',
      width: 150,
      align: 'left',
      sorter: true
    },
    {
      title: '部署名',
      dataIndex: 'departmentName',
      key: 'departmentName',
      width: 250,
      align: 'left',
      sorter: true
    },
    {
      title: '状態',
      dataIndex: 'isActive',
      key: 'isActive',
      width: 120,
      align: 'center',
      sorter: true,
      render: (isActive: boolean) => (
        <Badge
          status={isActive ? 'success' : 'default'}
          text={isActive ? 'アクティブ' : '非アクティブ'}
        />
      )
    },
    {
      title: '更新日時',
      dataIndex: 'updateAt',
      key: 'updateAt',
      width: 180,
      align: 'right',
      sorter: true,
      defaultSortOrder: 'descend',
      render: (date: string) => (date ? new Date(date).toLocaleString('ja-JP') : '-')
    },
    {
```

```
    title: '操作',
    key: 'action',
    width: 200,
    align: 'center',
    fixed: 'right',
    render: (_, record) => (
      <Space size="small">
        <Button
          type="default"
          size="small"
          icon={<EditOutlined />}
          onClick={() => {
            setEditingDepartmentCd(record.departmentCd);
            setIsEditDialogOpen(true);
          }}
        >
          編集
        </Button>
        <Button
          type="default"
          size="small"
          style={
            record.isActive
              ? {
                  color: token.colorError,
                  borderColor: token.colorError
                }
              : {
                  color: token.colorSuccess,
                  borderColor: token.colorSuccess
                }
          }
          onClick={() => handleStatusToggle(record.departmentCd, !record.isActive)}
        >
          {record.isActive ? '非アクティブ化' : 'アクティブ化'}
        </Button>
      </Space>
    )
  }
];

// テーブル変更ハンドラ
const handleTableChange = (
  newPagination: TablePaginationConfig,
  filters: any,
  sorter: any
) => {
  if (sorter.field) {
    setSortField(sorter.field);
    setSortOrder(sorter.order === 'ascend' ? 'asc' : 'desc');
```

```
    }
    setPagination({
      current: newPagination.current || 1,
      pageSize: newPagination.pageSize || 20,
      total: pagination.total
    });
  };

  return (
    <div style={{ padding: 24 }}>
      <h2>部署管理</h2>

      {/* ツールバー */}
      <Space style={{ marginBottom: 16, width: '100%', justifyContent: 'space-between' }}>
        <Space>
          <Input
            placeholder="部署CD、部署名で検索"
            prefix={<SearchOutlined />}
            style={{ width: 250 }}
            value={searchText}
            onChange={(e) => setSearchText(e.target.value)}
            allowClear
          />
          <Select
            placeholder="ステータス"
            style={{ width: 150 }}
            value={statusFilter}
            onChange={setStatusFilter}
            allowClear
            options={[
              { label: 'アクティブ', value: true },
              { label: '非アクティブ', value: false }
            ]}
          />
        </Space>

        <Space>
          <Button
            icon={<DownloadOutlined />}
            onClick={handleDownloadCsv}
          >
            CSV
          </Button>
          <Upload
            beforeUpload={handleUploadCsv}
            showUploadList={false}
            accept=".csv"
          >
            <Button icon={<UploadOutlined />}>CSV一括登録</Button>
          </Upload>
```

```
          <Button
            type="primary"
            icon={<PlusOutlined />}
            onClick={() => setIsAddDialogOpen(true)}
          >
            追加
          </Button>
        </Space>
      </Space>

      {/* テーブル */}
      <Table
        columns={columns}
        dataSource={data}
        rowKey="departmentCd"
        loading={loading}
        pagination={{
          current: pagination.current,
          pageSize: pagination.pageSize,
          total: pagination.total,
          showSizeChanger: true,
          showTotal: (total) => `全 ${total} 件`,
          pageSizeOptions: ['10', '20', '50', '100']
        }}
        scroll={{ x: 1000 }}
        onChange={handleTableChange}
      />

      {/* ダイアログ */}
      <DepartmentAddDialog
        open={isAddDialogOpen}
        onClose={() => setIsAddDialogOpen(false)}
        onSuccess={fetchData}
        organizationId={organizationId}
      />

      <DepartmentEditDialog
        open={isEditDialogOpen}
        onClose={() => {
          setIsEditDialogOpen(false);
          setEditingDepartmentCd(null);
        }}
        onSuccess={fetchData}
        departmentCd={editingDepartmentCd}
        organizationId={organizationId}
      />
    </div>
  );
}
```

9. テスト要件

9.1. 単体テスト

テストケース

- データ取得機能のテスト
- 検索機能のテスト（デバウンス動作含む）
- フィルタリング機能のテスト
- ソート機能のテスト
- ページネーション機能のテスト
- ステータス切替機能のテスト
- CSV入出力機能のテスト

9.2. 統合テスト

テストシナリオ

1. 画面初期表示（一覧取得）
2. 検索実行と結果表示
3. ステータスフィルタと結果表示
4. ソート実行と結果表示
5. ページ遷移
6. 部署追加（ダイアログ連携）
7. 部署編集（ダイアログ連携）
8. ステータス切替（確認ダイアログ含む）
9. CSVダウンロード
10. CSVアップロード（成功・失敗含む）

10. パフォーマンス考慮事項

10.1. データ取得の最適化

- ページネーションによるデータ量制限
- 検索のデバウンス処理（500ms）
- 不要な再レンダリングの抑制

10.2. メモ化

```
// 高コストな計算はuseMemoで最適化
const filteredData = useMemo(() => {
  return data.filter(/* フィルタ処理 */);
}, [data, searchText, statusFilter]);

// コールバック関数はuseCallbackで最適化
const handleEdit = useCallback((departmentCd: string) => {
  setEditingDepartmentCd(departmentCd);
  setIsEditDialogOpen(true);
}, []);
```

---

11. アクセシビリティ

11.1. キーボード操作

- Tab: 次の要素へフォーカス移動
- Shift + Tab: 前の要素へフォーカス移動
- Enter: ボタン実行、行選択
- Space: チェックボックス切り替え
- 矢印キー: テーブル内ナビゲーション

11.2. スクリーンリーダー対応

- テーブルに適切な `aria-label` を設定
- ボタンに明確なラベルを設定

- ローディング状態を `aria-live` で通知
- エラーメッセージを `role="alert"` で通知

---

12. エラーハンドリング

12.1. API エラー

エラーコード	表示メッセージ	対処方法
401	「認証エラーが発生しました。再ログインしてください」	ログイン画面へリダイレクト
403	「この操作を実行する権限がありません」	エラー表示のみ
404	「データが見つかりません」	エラー表示のみ
500	「サーバーエラーが発生しました。しばらく経ってから再試行してください」	エラー表示のみ
Network	「ネットワークエラーが発生しました。接続を確認してください」	再試行ボタン表示

12.2. ユーザーアクション失敗時

- CSVアップロード失敗: エラー詳細をモーダルで表示
- ステータス切替失敗: エラーメッセージをToastで表示
- データ取得失敗: エラーメッセージとリトライボタンを表示

---

13. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-01-23	初版作成	Development Team